睿尔曼智能 CRM — 风险识别与应对

> **版本**: v1.0
> **日期**: 2026-03-04
> **依赖文档**: 01~09 全部文档交叉审查
> **定位**: 识别系统方案、硬件、测试、开发计划、自动化流水线中的潜在问题和风险，给出应对措施与行动计划
> **审查方法**: 逐文档精读 → 交叉比对 → 分级分类 → 给出行动项

风险总览表

> 🔴 P0 阻塞 | 🟠 P1 高风险 | 🟡 P2 中风险 | 🔵 P3 建议 | ⚪ 一致性
> **合计 44 项**: 🟢 已解决 14 | 🟠🟡🔵 待处理 22 | ⭕ 已废弃 1 | ⚪ 待修正 8

编号	等级	风险名称	截止时间	状态
R-001	🔴 P0	API Key / 凭据明文硬编码	S1 Day 1	🟢 已解决
R-002	🔴 P0	模型文件缺失 — Qwen3-30B-A3B 不完整	S1 Day 1	✅ 已解决
R-003	🔴 P0	~~自举悖论~~ — aider 已覆盖核心循环，Orchestrator 简化为薄调度层	S1 Day 1	🟢 已解决
R-004	⚫ P1	~~05-测试方案未与架构迁移同步~~ — 已全文修正 L20 → orchestrator	S1 前	🟢 已解决
R-005	🟠 P1	单人 + 6 机器 + 4 周 = 极高执行风险	S1 前	🟠 待处理
R-006	🟠 P1	yibuapi.com 单点依赖	S1 前	🟠 待处理
R-007	🟠 P1	~~aliyun-agent 到内网网络未确认~~ — 已移除 aliyun-agent，全部业务内网运行	S1 前	🟢 已解决
R-008	🟠 P1	~~Embedding 模型未规划部署~~ — 一期用阿里云百炼 Embedding API	S5 前	🟢 已解决
R-009	🟠 P1	AutoReview 改用云端 API（已决策）	S1 前	🟢 已解决
R-010	🟠 P1	~~钉钉集成前置条件未确认~~ — 已创建机器人（Stream 模式），内网已打通	S1 前	🟢 已解决
R-011	🟠 P1	4 份关键文档尚未编写	S2 末尾	🟠 待处理
R-012	🟡 P2	mac_min_8T 角色过载	本 Sprint	🟡 待处理
R-013	🟡 P2	~~Git 分支冲突风险被低估~~ — 已缓解（串行提交 + 人工确认）	S1-S2	🟢 已缓解
R-014	🟡 P2	S1-2 "已完成" 状态不清晰	S1 前	🟡 待处理
R-015	🟡 P2	Redis 无持久化配置	S1-S2	🟡 待处理
R-016	🟡 P2	Git Bare Repo 无备份	S1-S2	🟡 待处理
R-017	🟡 P2	Cloud Judge API 成本与可用性	S3 前	🟡 待处理
R-018	~~🔵 P3~~	~~升级 vLLM B~~ 已废弃（vLLM B 已移除）	~~	🟢 已废弃
R-019	🔵 P3	~~优先开发 Orchestrator Dashboard~~ — 已缓解（钉钉交互查询 + 终端看板）	S2 末尾	🟢 已缓解
R-020	🔵 P3	~~提前准备 Docker Compose~~ — 已采纳（S1 编写 docker-compose.yml）	S1	🟢 已采纳
R-021	🔵 P3	增加磁盘空间监控	S1-S2	🔵 Backlog
R-022	🔵 P3	考虑一期减少 Agent 数量	S1 前	🔵 Backlog
R-023	🟠 P1	数据安全合规 — PIPL 未纳入设计	S3 前	🟠 待处理

编号	等级	风险名称	截止时间	状态
R-024	🟠 P1	ERP(U9) 集成方案缺失 — 分期未确认	S3 前	🟠 待处理
R-025	🟠 P1	多语言 / 国际地图需求被忽略	S3 前	🟠 待处理
R-026	🟡 P2	Redis Stream 无 Consumer Group / 回压设计	S3 中	🟡 待处理
R-027	🟡 P2	SSL/HTTPS 未规划 — 钉钉要求 HTTPS	S3 前	🟡 待处理
R-028	🟡 P2	gateway 无公网 IP — 钉钉回调不可达	S3 Day 1	🟡 待处理
R-029	🟡 P2	Agent 自研 — Tool Calling 方案缺失	S2 末尾	🟡 待处理
R-030	🟡 P2	Qwen3-30B-A3B 多步 Tool Calling 能力存疑	S3 Day 2	🟡 待处理
R-031	🟡 P2	PostgreSQL 在 macOS — Docker 化困难	S6 末尾	🟡 待处理
R-032	🟡 P2	Celery Worker 跨网络消费 Redis	S3 中	🟡 待处理
R-033	🟡 P2	缺少错误处理和用户体验降级方案	S3 前	🟡 待处理
R-034	🟡 P2	02 智能扩展与一期范围不对齐	S3 前	🟡 待处理
R-035	🔵 P3	缺少 Agent 可观测性 (Tracing)	S3 中	🔵 Backlog
R-036	🔵 P3	Git 裸仓库位置与文档不一致	立即	🔵 待修正
C-001	⚪	测试执行位置描述不一致	S1 前	⚪ 待修正
C-002	⚪	vLLM 实例编号约定不一致	S1 前	⚪ 待修正
C-003	⚪	post-receive hook 逻辑版本不一致	S1 前	⚪ 待修正
C-004	⚪	08 §2.5 YAML 缩进错误	S1 前	⚪ 待修正
C-005	⚪	04 §6.1 缺少 orchestrator 角色	S1 前	⚪ 待修正
C-006	⚪	Judge 评分维度不一致	S1 前	⚪ 待修正
C-007	⚪	R-016 中 Git 裸仓库位置描述错误	立即	⚪ 待修正
C-008	⚪	02 智能扩展需求缺少分期标注	S3 前	⚪ 待修正

目录

1. 风险分级标准
2. P0 阻塞级问题（必须立即处理）
3. P1 高风险问题（需在开发前解决）
4. P2 中风险问题（需要关注的设计问题）
5. P3 优化建议
   5b. 补充风险项（v1.1 Review 新增）
6. 文档间交叉一致性问题
7. 优先行动计划与跟踪表
8. 风险全景统计

1. 风险分级标准

级别	含义	响应时限	颜色
P0 阻塞	直接阻塞开发或导致安全事故，不修复无法启动 Sprint	**立即**	🔴
P1 高风险	不处理将在 1-2 个 Sprint 内暴发，导致返工或目标无法达成	**S1 开始前**	🟠
P2 中风险	当前不阻塞但会影响系统健壮性或运维效率	**本 Sprint 内**	🟡
P3 优化	改进建议，提升效率或质量	**排入 Backlog**	🟢

2. P0 阻塞级问题（必须立即处理）

R-001 🔴 API Key / 凭据明文硬编码在设计文档中

维度	内容
涉及文档	08-自动化开发流水线.md (§1.5, §2.5, §3.2, §13.5, dispatcher 源码 共 5+ 处)、03-硬件清单.md (5 台机器登录密码)、09-测试机搭建计划.md (§2.3 PG 密码 `test_pass`)
风险描述	yibuapi.com 的 API Key 在 08 文档中以明文出现 5+ 次（已修复）。03 文档含全部 5 台机器的 SSH 登录密码（已修复）。09 文档含数据库密码（已修复）。这些凭据已迁移到 `.env` 文件，文档中替换为 `${变量名}` 占位符。
影响范围	全部 5 台机器的安全性、Claude Opus 4.6 编码 API 的资金安全、测试数据库安全
发生概率	高 — Git 仓库是团队协作的核心，扩展人员或迁移仓库时极易泄漏
应对措施	1. 所有凭据替换为环境变量引用（如 `${OPENAI_API_KEY}` 、 `${PASSWORD_ORCHESTRATOR}` ）✅ 2. 各机器的实际凭据写入 `.env` 文件，不入 Git✅ 3. `.env.example` 提供模板✅ 4. `.gitignore` 排除 `.env` ✅
责任人	项目负责人
截止日期	S1 Day 1 —— 已完成
状态	🟢 已解决（2026-03-04）

R-002 🔴 模型文件缺失 — Qwen3-30B-A3B 不完整

维度	内容
涉及文档	04-系统概要设计.md §7.1、08 §1.4 (vLLM A/C 均依赖此模型)
风险描述	`models/Qwen3-30B-A3B/` 目录下共 16 个分片文件（model-00001 至 model-00016），其中 `model-00011-of-00016.safetensors` 缺失。vLLM 加载需要全部分片，缺失任何一个都将报错无法启动，**直接阻塞 vLLM A (4090) 的部署**。
影响范围	Agent 推理 + 测试 Agent 推理（vLLM A）全部不可用
发生概率	已确认 — 当前文件系统中缺失该分片
应对措施	1. 立即从 HuggingFace 或原始源重新下载缺失分片： `huggingface-cli download Qwen/Qwen3-30B-A3B model-00011-of-00016.safetensors --local-dir ~/models/Qwen3-30B-A3B/` 2. 下载完成后校验完整性：对照 `model.safetensors.index.json` 中的 weight_map 确认全部 16 个分片存在
责任人	项目负责人
截止日期	立即

维度	内容
状态	🟢 已解决（4090 vLLM A 已正常运行，aliyun-agent 已移除无需同步）

R-003 ~~自举悖论~~ — aider 已覆盖核心循环，Orchestrator 简化为薄调度层

维度	内容
涉及文档	08-自动化开发流水线.md §2、07-Sprint任务卡.md S1-2 W0 列、04-系统概要设计.md §13
原始风险	Orchestrator 被设计为重型定制系统（DocParser + TaskEngine + Dispatcher + Judge + StateMachine，预估 2000-3000 行 Python），导致"自举悖论"——自动化系统本身需要手工编码，S1-2 无法享受自动化红利。
解决方案	重新评估后发现 **aider 本身已是一个完整的自动编码循环**（读取上下文 → LLM 生成代码 → 自动写入 → git commit → 运行 lint/test → 失败自动修复 → 重试直到通过）。Orchestrator 不需要从零实现这些能力，只是一个 **薄调度层**：告诉 aider 在哪台机器上做什么任务，然后收集 exit code。核心逻辑约 200 行 shell/Python 脚本，S1 Day 1-2 即可完成最小可用版本。自举悖论不再成立。
架构变更	08 文档 §2 的伪代码保留为逻辑设计参考，但实际实现路径为： 1. **S1 Day 1-2**：编写 `orchestrator.sh` 调度脚本（~200 行），能 SSH 到 5 台机器运行 aider 2. **S3-4 渐进增强**：加入 DocParser（云端 API 一次调用）、钉钉通知、多模型 Judge 3. **S5+**：按需加入 Redis 任务队列、并发调度等高级功能
影响范围	08 §2 Orchestrator 架构说明需注明 aider 已有能力边界；04 §13、06 §1.3 同步简化 Orchestrator 描述
责任人	项目负责人
截止日期	S1 Day 1
状态	🟢 已解决（Orchestrator 重新定位为薄调度层，自举悖论消解）

3. P1 高风险问题（需在开发前解决）

R-004 🟢 ~~05-测试方案大量内容未与架构迁移同步~~ — 已解决

维度	内容
原始问题	05 §2 已声明“测试迁移到 orchestrator 本地执行”，但后续 §5.1/§5.2/§5.3/§6.1/§6.3/§7.1/§7.2/§9/§10 大量章节仍引用旧 "L20" 架构。
解决方案	全文批量修正，共计 30+ 处 L20 引用统一迁移至 orchestrator。具体包括：§2 测试环境描述（标题/配置表/SSH/服务栈/数据隔离）、§5 测试金字塔（运行位置/依赖/原理说明）、§6 流水线触发图 + test-runner 脚本、§7 非功能测试（环境表/locust拓扑/连接说明）、§9-10 搞建排期/资源表。
保留的 L20 引用	版本变更说明、迁移说明、旧方案对比 (§4.1)、成本对比 (§4.7) — 这些均为历史上下文。~~aliyun-agent Staging 硬件~~ 已移除。
关联文档	05-测试方案与计划.md v0.5
状态	🟢 已解决

R-005 🟠 单人团队 + 5 机器运维 + 4 周交付 = 极高执行风险

维度	内容
涉及文档	06-总体开发计划.md、07-Sprint任务卡.md、03-硬件清单.md

维度	内容
风险描述	项目为 1 人团队，需同时管理：(1) 5 台机器的硬件运维（SSH、GPU 驱动、防火墙、磁盘空间）；(2) 庞大的软件栈（PostgreSQL + Redis + vLLM×1 + Nginx + Celery + Prometheus + Grafana + aider×5）；(3) 一期开发产出（6 Agent + 11 Tool + 4 REST API 组 + 6 Vue3 页面 + 300+ 测试）；(4) 9 份现有文档维护 + 4 份待编写新文档；(5) 外部依赖协调（钉钉 AppKey、3 个云端 Judge API Key、yibuapi.com 可用性）。任何一台机器的故障都会阻塞开发。
影响范围	项目整体交付时间线
发生概率	高 — 5 台异构机器的运维本身就需要大量时间
应对措施	1. **砍一期非核心功能**：Vue3 看板可降级为极简单页面（1-2 页而非 6 页） 2. **钉钉互动卡片 / 审批流推到二期**，一期只做机器人对话 3. 接受"Agent 不完美但可用"的质量基线，Prompt 调优不追求完美 4. **优先搭建 Orchestrator + 自动化**，让机器分担编码工作量 5. 准备"功能降级交付矩阵"：核心 3 Agent（Supervisor + 销售助手 + 线索评分）必保，其余 3 个 Agent 视进度取舍
责任人	项目负责人
截止日期	S1 前确认砍减方案
状态	🟠 待处理

R-006 🟠 第三方 API 网关（yibuapi.com）单点依赖

维度	内容
涉及文档	08-自动化开发流水线.md §1.5、§3.2
风险描述	全部 5 台机器的 aider 编码统一通过 yibuapi.com 转发 Claude Opus 4.6。该第三方：(1) 非 Anthropic 官方，无 SLA 保证；(2) 可能随时调价、限流或下线；(3) 如果不可用，**所有自动化编码停摆**。编码是流水线最核心的环节，如果停摆将导致每日 50-100 个任务归零。
影响范围	全部 5 台机器的自动化编码能力
发生概率	中 — 第三方服务中断概率不低，尤其在促销/维护时
应对措施	1. **准备 fallback 编码模型配置**：本地 Qwen3-30B-A3B + aider（效果弱于 Claude 但可兜底），或备用 API 网关 2. **估算月成本并设预警**：50-100 task/day × ~4K tokens/task × Claude 单价 ≈ 每月 $XXX，设置消费上限 3. 在 Orchestrator 的 dispatcher 中实现**自动降级**：yibuapi.com 连续 3 次超时 → 切本地 vLLM 编码 4. 考虑同时配置一个备用 Claude API 供应商
责任人	项目负责人
截止日期	S1 前
状态	🟠 待处理

R-007 🟠 ~~aliyun-agent 到内网的网络连通性未确认~~ — 🟢 已解决（aliyun-agent 已移除）

维度	内容
涉及文档	~~09-测试机搭建计划.md~~ (已废弃)、03-硬件清单.md §6 (已移除云端部分)、04/05/06/07/08 已同步更新
风险描述	~~多处功能假设云端 aliyun-agent 可以 SSH 到内网机器，但 172.16.x.x 为私有地址，从公网无法直达。~~ **已解决：彻底移除 aliyun-agent，全部业务在 5 台内网机器上运行。**
解决方案	不再尝试打通云端与内网的网络（VPN/FRP），而是直接从架构中移除 aliyun-agent： 1. 硬件从 6 台缩减为 5 台（全内网 172.16.x.x） 2. Staging 预发布验收迁移至 orchestrator（独立 DB sirius_crm_staging + Redis DB 2 + 独立配置）

维度	内容
	3. vLLM C 移除，Agent 推理统一调用 4090 vLLM A 4. 文档 03～09 已全部同步更新，09 整篇废弃
责任人	项目负责人
解决日期	v1.3
状态	🟢 已解决

R-008 🟠 ~~Embedding 模型未规划部署~~ — 🟢 已决策（一期云端 API）

维度	内容
涉及文档	04-系统概要设计.md §7.1 (选型 BGE-M3 或 Qwen-Embedding)、§8.3 Knowledge 表 (vector(1024))
风险描述	~~04 §7.1 在技术选型中明确提到使用 BGE-M3 或 Qwen-Embedding 做向量嵌入，但未规划部署位置和时间。~~ **已决策。**
已决策方案	**一期：阿里云百炼 Embedding API（text-embedding-v3）** - 零部署、零 GPU 占用，直接调用云端 API 生成 1024 维向量 - 一期知识量小（百千条），每天调用百次级，成本几乎为零（几元/月） - 阿里云百炼已在使用（Judge 评估走百炼 Qwen-Max），无需额外开通 - Knowledge 表 `vector(1024)` 字段设计不变，pgvector HNSW 索引不变 **二期预留**：知识量达万级/调用频率达千次/天时，迁移到本地 BGE-M3（4090 上与 vLLM A 并存，仅需 2-4GB 显存）
影响范围	知识库语义检索功能、客户洞察 Agent 的 RAG 能力
责任人	项目负责人
决策日期	v1.4
状态	🟢 已解决（一期云端 API，二期视知识量迁移本地）

R-009 🟠 AutoReview 改用云端 API（已决策）

维度	内容
涉及文档	08-自动化开发流水线.md §4.1-4.2 (三层自动 Review)、04 §7.1 (Qwen3-30B-A3B, MoE 激活3B)
风险描述	原方案中 vLLM B 跑的是 Qwen3-30B-A3B（MoE 架构，实际激活仅 3B 参数），而编码使用的是 Claude Opus 4.6（顶级模型）。用远弱于编码模型的模型来审查代码，存在漏判和误报风险。
已决策	**AutoReview L2/L3 和 DocParser 统一走云端 API**（yibuapi.com / Claude API），Review 频率低于编码，成本增加可控。 同时移除 orchestrator 上的 vLLM B 实例，释放 GPU 资源。
影响范围	自动 Review 的有效性、代码质量保证
发生概率	已通过云端 API 解决
应对措施	✅ AutoReview L2/L3 和 DocParser 改用云端 API（强模型），已在 05/08 文档中更新
责任人	项目负责人
截止日期	S1 前确认策略 —— 已确认
状态	🟢 已解决（迁移云端 API）

R-010 🟠 ~~钉钉集成前置条件未确认~~ — 🟢 已解决（Stream 模式）

维度	内容
涉及文档	04-系统概要设计.md §14 待确认清单 #2、§7.5 钉钉集成选型
风险描述	~~钉钉应用未注册，无 AppKey/AppSecret，审批流程不可控。~~ **已解决。**
已完成工作	1. 钉钉开发者应用已创建，获得 AppKey/AppSecret 2. 采用 **Stream 模式**（长连接）而非 HTTP 回调模式，无需公网 URL / SSL 证书 / 域名 3. 已与内网打通，能够正常获取数据 4. 测试环境机器人已可用
影响范围	钉钉联调关键路径已打通，R-027、R-028 同步解决（Stream 模式无需公网回调）
责任人	项目负责人
解决日期	v1.5
状态	🟢 已解决

R-011 🟠 4 份关键文档尚未编写

维度	内容
涉及文档	04-系统概要设计.md 附录 A 文档规划
风险描述	以下 4 份文档标记为"待编写"，但它们是后续 Sprint 的**阻塞依赖**：(1) **Agent 详细设计**（10） — S3-4 开始写 Agent 前必须有 Prompt 设计、Tool 定义、Agent 间协议；(2) **数据库设计**（11） — S1-2 创建 Schema 需要完整表结构；(3) **钉钉集成方案**（12） — S3-4 联调需要方案；(4) **部署方案**（13） — S7-8 切运行模式需要 docker-compose.yml 等。
影响范围	S1-2 Schema 创建、S3-4 Agent 开发和钉钉联调、S7-8 部署
发生概率	必然 — 目前为空白
应对措施	1. **数据库设计**：S1 Day 2 前完成（阻塞 Schema 创建） 2. **Agent 详细设计**：S2 末尾前完成（阻塞 S3-4 Agent 编码） 3. **钉钉集成方案**：S2 末尾前完成（阻塞 S3-4 联调） 4. **部署方案**：S6 末尾前完成（阻塞 S7-8 上线切换）
责任人	项目负责人
截止日期	见上述各子项
状态	🟠 待处理

4. P2 中等风险问题（需在相关 Sprint 前解决）

R-012 🟡 mac_min_8T 角色过载

维度	内容
涉及文档	03-硬件清单.md §2、04-系统概要设计.md §5、08-自动化开发流水线.md §2.4
风险描述	mac_min_8T (M4 Pro 64GB) 同时承担：(1) PostgreSQL + pgvector 数据库、(2) Redis 缓存（含 Stream 事件总线、对话上下文）、(3) Celery Worker、(4) 前端 Vue3 开发构建。~~(原有 Ollama Embedding 推理已移除 — R-008 决议一期走阿里云百炼 API，二期部署在 4090)~~。macOS 不支持 Docker GPU 透传（Metal 不兼容），未来 Docker 化时需要特殊处理。
影响范围	数据库可用性、前端构建效率

维度	内容
发生概率	中偏低 — 去掉 Embedding 推理后负载降低，64GB 内存足够，但 macOS 长时间运行稳定性一般
应对措施	1. PostgreSQL 启用 WAL 归档 + 定期 pg_dump 到 orchestrator 2. 考虑将 Redis 迁移到 orchestrator（128GB 内存更宽裕） 3. 设置 macOS 自动重启脚本（launchd），服务崩溃自动恢复
责任人	项目负责人
截止日期	S1 中
状态	🟡 待处理

R-013 🟡 ~~Git 分支冲突风险被低估~~ — 🟢 已缓解

维度	内容
涉及文档	08-自动化开发流水线.md §5.2 自动 commit + push 流程、§5.4 分支策略
原始风险	~~08 设计了多机并行编码（同时最多 5 台 aider 提交），共享文件（requirements.txt、config.py 等）必然冲突，冲突处理"等待人工"会导致任务永久挂起。~~
实际评估	该风险被严重高估：(1) 1 人团队不会真正 5 台机器同时无人值守并行 push；(2) 按机器划分子目录（agent/ / crm/ / deploy/nginx/ / deploy/monitor/）已大幅降低冲突概率；(3) 共享文件修改（新增依赖、注册 Agent）是低频事件，非每次提交都发生。
已采纳方案	1. **串行提交 + 人工确认**：aider 编码后不自动 push，由开发者手动 review 后 push 2. **共享文件单点管理**： `requirements.txt` 、 `config.py` 等共享文件仅在 mac_min_8T（CRM 主机）上修改，其他机器不碰 3. **单分支策略不变**：一期继续用 main 单分支，按子目录隔离 + 串行提交即可保证无冲突 4. ~~分布式锁（Redis）、自动 merge 脚本~~ — 不需要，过度工程
影响范围	多机开发流程简化，去掉不必要的并行冲突处理复杂度
责任人	项目负责人
解决日期	v1.6
状态	🟢 已缓解

R-014 🟡 S1-2 "已完成" 状态不清晰

维度	内容
涉及文档	06-总体开发计划.md、07-Sprint任务卡.md
风险描述	06 和 07 将 S1-2 标记为"✅ 已完成"，但：(1) Orchestrator 薄调度层（~200 行脚本）是否已有可运行的代码？如果没有，S1-2 的"完成"只是文档和环境准备；(2) workspace 中未见 Orchestrator 的代码仓库；(3) 各机器基础环境是否已就绪（vLLM A、PostgreSQL、Redis 等）。"已完成"是需要重新验证的。
影响范围	S3-4 的启动条件
发生概率	中 — 可能只是文档完成而非软件环境 ready
应对措施	1. 创建 S1-2 **交付物检查清单**：Git Bare Repo 可用？vLLM A (4090) 可服务？PostgreSQL (mac_min_8T) 可连接？aider 可编码？钉钉 Stream 机器人可收发？ 2. 逐项验证，未通过项补入 S3 前置任务 3. 将检查清单结果记入本文档附录
责任人	项目负责人
截止日期	S3 Day 1 前

维度	内容
状态	🟡 待处理

R-015 🟡 Redis 无持久化配置

维度	内容
涉及文档	04-系统概要设计.md §7.3
风险描述	Redis（mac_min_8T 上）当前无 RDB/AOF 持久化配置。Redis 同时承担：(1) Celery Broker（任务队列）、(2) Agent 会话上下文缓存、(3) Orchestrator 任务调度状态、(4) Stream 事件总线。一旦 Redis 进程重启或 OOM，**所有进行中的 Celery 任务丢失**，Orchestrator 的任务状态也会断裂。
影响范围	任务队列可靠性、会话连续性
发生概率	中 — macOS 上 Redis 长期运行有内存泄漏风险
应对措施	1. 至少开启 `appendonly yes` (AOF) 持久化 2. Celery 建议配置 `task_acks_late=True` + `worker_prefetch_multiplier=1` 防止任务丢失 3. 关键状态同时写入 PostgreSQL（双写），Redis 作为加速层而非唯一存储
责任人	项目负责人
截止日期	S1 中
状态	🟡 待处理

R-016 🟡 Git Bare Repo 无备份

维度	内容
涉及文档	08-自动化开发流水线.md §5.3-§5.4
风险描述	Git Bare Repo 位于 mac_min_8T 单机上（08 §5.3 post-receive hook），是全部代码和自动化流水线的**唯一源头**。一旦硬盘故障，全部代码丢失。目前没有任何备份或镜像策略。
影响范围	全部代码和开发历史
发生概率	低 — 但后果极其严重（灾难性）
应对措施	1. 每日 cron 将 bare repo `git clone --mirror` 到 orchestrator 2. 每周推送到 GitHub/GitLab 私有仓库做异地备份 3. 在 post-receive hook 中增加备份触发
责任人	项目负责人
截止日期	S1 中
状态	🟡 待处理

R-017 🟡 Cloud Judge API 成本与可用性风险

维度	内容
涉及文档	05-测试方案与计划.md §4.5
风险描述	三模型 Judge（GPT-4.1 40%、Qwen-Max 35%、DeepSeek-V3 25%）用于测试打分：(1) 每个测试用例调 3 个 API，300+ 测试 × 3 = 900+ API 调用 / 次全量测试；(2) 每次 API 调用含长 System Prompt + 测试输出，token 消耗约 2-4K/次；(3) 3 家 API 任一不可用就导致评分不完整；(4) 月成本可能达到 $50-200 取决于测试频率。
影响范围	测试自动化频率、月度成本

维度	内容
发生概率	中 — API 可用性通常较好，但成本可能超预期
应对措施	1. **设置每日测试频率上限**（如：全量测试每日 ≤ 2 次，PR 触发只跑相关模块） 2. Judge 结果缓存：相同输入 + 相同评分标准不重复调用 3. 准备降级方案：如只用 1 个模型（Qwen-Max，国内最稳定最便宜）做快速测试
责任人	项目负责人
截止日期	S3 前
状态	🟡 待处理

5. P3 优化建议（可按优先级逐步实施）

R-018 🟢 ~~升级 vLLM B 至 Qwen3.5-35B-A3B~~ — 已废弃（vLLM B 已移除）

维度	内容
涉及文档	04-系统概要设计.md §7.1、08-自动化开发流水线.md §4.1
原风险描述	workspace 中已下载 Qwen3.5-35B-A3B 模型文件，建议升级 vLLM B。
废弃原因	vLLM B 已从架构中移除。AutoReview/DocParser 改用云端 API，Agent 测试推理统一调用 4090 vLLM A。此项不再适用。
优先级	已废弃

R-019 🟢 ~~优先开发 Orchestrator Web Dashboard~~ — 已缓解（三层渐进式可观测）

维度	内容
涉及文档	08-自动化开发流水线.md §6
风险描述	目前 Orchestrator 的可观测性为 0 — 无法实时看到任务分配、aider 编码状态、Review 结果、测试报告。08 §6 提到了 Web Dashboard 但标记为"后续迭代"。单人团队运维 5 台机器，**没有仪表盘等于盲飞**。
应对方案	三层渐进式可观测（总投入 ~3h，Web Dashboard 降级为可选待办）： **层级 1：钉钉交互式查询（S2，~2h）** — 复用已有 Stream 模式钉钉机器人，添加指令响应：`状态`（所有任务状态表）、`机器`（5 台 SSH + vLLM health）、`日志 <task_id>`（最近 20 行日志）、`卡住`（>30min 无进展的任务）。手机随时主动查询，~100 行代码。 **层级 2：终端实时看板（S2 末尾，~1h）** — `live_status.py + watch -n 30` 刷新，显示机器健康/当前任务/最近 Review 评分。读 Redis + SSH ping + rich 格式化，~80 行代码。开发时 tmux 常驻。 **层级 3：Web Dashboard（待办/可选，S7-S8）** — FastAPI + Jinja2 单文件，复用层级 2 数据采集逻辑，~150 行。AI 辅助 ~30min 可完成。对单人团队非刚需，层级 1+2 已覆盖 100% 可观测需求。
决策理由	层级 1（钉钉）= 手机随时查，覆盖 80% 场景；层级 2（终端）= 开发时持续监控，覆盖 20%；层级 3（Web）= 锦上添花，多一个 FastAPI 进程 = 多一个维护负担，单人团队无实质提升
优先级	层级 1+2 强烈建议（S2），层级 3 可选待办
状态	🟢 已缓解 — 层级 1+2 纳入 S2 计划，层级 3 列为待办

R-020 ⚠ 提前准备 ~~Docker Compose~~ — 已采纳（S1 编写 docker-compose.yml）

维度	内容
涉及文档	06-总体开发计划.md S7-8、04-系统概要设计.md §11

维度	内容
风险描述	部署方案（doc 13）列为"待编写"，S7-8 才做 Docker 化。但如果开发期间在裸机上跑，到 S7-8 迁移到 Docker 时可能遇到大量环境差异。
应对方案	S1 编写基础 `docker-compose.yml`，开发期间 Docker 跑依赖服务： - **PostgreSQL 16** + pgvector 扩展（数据持久化到 volume） - **Redis 7**（AOF 持久化，配合 R-015） 应用本身（Django 后端、Agent、Celery）保持裸机开发，S7-S8 只需增加 vLLM、Nginx、前端容器。 docker-compose.yml 纳入 Git 仓库，所有机器共享同一份依赖配置。
优先级	必须 — 已确认为 S1 必做项
状态	🟢 已采纳 — 纳入行动计划 #14

R-021 🔵 增加磁盘空间监控

维度	内容
涉及文档	08-自动化开发流水线.md §2.5 资源规划、03-硬件清单.md
风险描述	vLLM A 模型文件占据大量磁盘（Qwen3-30B-A3B 约 30GB），加上 Git 历史、日志文件、PostgreSQL WAL。4090（模型存储）和 mac_min_8T（DB + Git）可能在 S5-6 后磁盘紧张。~~原 orchestrator 上的 vLLM B 模型文件已移除。~~
建议	1. 各机器添加 cron `df -h` 检查，<20% 余量时告警 2. 定期清理旧 Git 分支和日志文件 3. 在 Prometheus 中添加 `node_filesystem_avail_bytes` 告警规则
优先级	建议

R-022 🔵 考虑一期减少 Agent 数量

维度	内容
涉及文档	04-系统概要设计.md §3.2（14 Agent 规划）、06-总体开发计划.md（一期 6 Agent）
风险描述	一期规划 6 个 Agent，每个 Agent 含 Prompt 设计 + Tool 实现 + 测试用例 + 联调。在 R-005 识别的执行风险下，6 个 Agent 可能做不完。
建议	将一期**核心保障**缩到 3 个 Agent：(1) Supervisor（路由必需）、(2) 销售助手（核心价值）、(3) 线索评分（差异化卖点）。其余 3 个（客户洞察、任务管理、知识检索）推到二期或作为 Plus 功能。
优先级	强烈建议 — 与 R-005 联动

5b. 补充风险项（v1.1 Review 新增）

> 以下风险项是 v1.1 版本交叉审查中新发现的遗漏，按优先级分级归入。

R-023 🟠 数据安全合规风险 — 个人信息保护法（PIPL）未纳入设计

维度	内容
涉及文档	01-基础需求 (客户/联系人/线索管理)、02-智能扩展 (§1.1 多渠道自动归集、§2.1 客户画像)、04-系统概要设计 §7.8

维度	内容
风险描述	CRM 系统处理大量个人信息（联系人姓名、电话、邮箱、拜访定位...），02 §1.1 还要求"自动抓取公开信息"、"工商信息补全"、"竞品动态监测"。中国《个人信息保护法》(PIPL) 要求：(1) 收集需明示同意；(2) 加工有最小必要原则；(3) 跨系统传输需评估。但**整个方案没有数据分类分级、用户同意流程、数据保留策略**的设计。 如果客户是上市公司或有合规要求的企业，这将是重大法律风险。
影响范围	法律合规、客户信任
发生概率	高 — PIPL 已生效，CRM 本身就是个人信息密集系统
应对措施	1. 在 04 文档中增加"数据安全与隐私"章节，定义数据分类（公开/内部/机密/极密） 2. 联系人创建时要求标注信息来源；自动抓取功能需在 UI/钉钉卡片中增加"是否授权采集"确认 3. 04 §8 数据库设计中增加 `data_source` 和 `consent_flag` 字段 4. 推迟"自动抓取公开信息"功能到合规评估完成后
责任人	项目负责人
截止日期	S3 前
状态	🟠 待处理

R-024 🟠 ERP (U9) 集成接口方案缺失 — 一期无法完全忽略

维度	内容
涉及文档	01-基础需求 (§系统集成-ERP: 14项对接)、04-系统概要设计 §5.4 工具定义 (integration 标注"二期")
风险描述	01 基础需求中 ERP(U9) 集成列了 **14 项双向同步**（产品同步、汇率、客户推送、订单同步、发货、出库、回款、发票、退货退款等），是 SOW 合同内容。04 §5.4 将 `erp_*` 全部标注为"二期"。但**01 需求没有明确划分一期二期**， 客户可能期望一期就有基础 ERP 对接。如果验收时客户认为订单/发货/回款的 ERP 同步是一期范围，将产生重大纠纷。
影响范围	客户验收、合同纠纷
发生概率	高 — SOW 中未明确分期
应对措施	1. **立即与客户确认一期验收范围**：明确哪些 ERP 集成在一期交付，哪些在二期 2. 建议一期至少实现**产品同步**（只读）+ **客户推送**（CRM→ERP），作为最小 ERP 对接 3. 将确认结果写入补充协议或会议纪要，避免口头承诺
责任人	项目负责人
截止日期	S3 前确认
状态	🟠 待处理

R-025 🟠 多语言 / 国际地图需求被完全忽略

维度	内容
涉及文档	01-基础需求 (§多语言及国际地图)、04-系统概要设计
风险描述	01 需求明确包含"支持多种语言（中英双语）"、"国际地图"、"海外客户地图定位和签到打卡"。但 04 系统概要设计中**完全未提及多语言和国际化**。如果客户有海外业务（从"机械臂"品类推测极可能），一期无法使用。 需确认此需求是否在一期范围内。
影响范围	客户验收、海外销售使用
发生概率	中 — 取决于客户确认
应对措施	1. 与客户确认国际化是否一期范围 2. 如为一期，Agent 的 Prompt 和 Tool 输出需预留多语言支持（i18n 字段设计）

维度	内容
	3. 如推二期，技术上建议一期前端 Element Plus 提前集成 i18n 框架，避免二期大面积重构
责任人	项目负责人
截止日期	S3 前确认
状态	🟠 待处理

R-026 🟡 事件总线 (Redis Stream) 无消费者确认与回压设计

维度	内容
涉及文档	04-系统概要设计 §3.1 (事件总线)、§5.5 (记忆系统)、§7.2 (Redis Stream)
风险描述	04 §3.1 将 Redis Stream 作为"Agent 的神经网络"，所有业务事件通过它分发。但未设计：(1) Consumer Group 设计（多 Agent 消费同一事件）；(2) 消息积压（back-pressure）策略——如果某 Agent 处理慢，Stream 是否设上限？；(3) Dead Letter 机制——消费失败的事件怎么处理？(4) Stream 数据淘汰策略（MAXLEN）。一旦主动型 Agent 数量增加（巡检+预警+催收+规划+晨报 = 5 个并发消费者），Redis 内存可能快速膨胀。
影响范围	Agent 主动行为可靠性、Redis 内存
发生概率	中 — S5-6 主动型 Agent 入场后暴露
应对措施	1. 在 S3-4 首个事件驱动 Agent 实现前，设计 Consumer Group 方案 2. 设置 `MAXLEN ~10000` 防止 Stream 无限增长 3. 消费失败的事件写入 PostgreSQL 的 dead_letter 表，供人工排查
责任人	项目负责人
截止日期	S3 中
状态	🟡 待处理

R-027 🟡 ~~SSL / HTTPS 未规划 — gateway 仅配置 HTTP~~ — 🟢 已解决（Stream 模式无需 HTTPS 回调）

维度	内容
涉及文档	04-系统概要设计 §6.1 (Nginx :80/:443 标注)、07-Sprint任务卡 S1-2 Day 3 (Nginx 基础配置)
风险描述	~~钉钉机器人的回调 URL 必须是 HTTPS，但无 SSL 配置方案。~~ **已解决：采用 Stream 模式（长连接），不需要 HTTP 回调 URL，SSL/HTTPS 不再是钉钉集成的阻塞项。**
说明	Stream 模式下，钉钉 SDK 主动从内网向钉钉服务器发起 WebSocket 长连接，不需要公网可达的回调 URL。但 Web 管理后台、Vue3 前端如需外部访问，SSL 仍建议配置（可推迟到二期）。
状态	🟢 已解决（钉钉场景）

R-028 🟡 ~~gateway 无公网 IP / 域名 — 钉钉回调不可达~~ — 🟢 已解决（Stream 模式无需公网）

维度	内容
涉及文档	03-硬件清单 §3 (gateway 172.16.14.215)、04-系统概要设计 §6.1-6.2
风险描述	~~gateway 为私有地址，钉钉回调需要公网可达的 HTTPS URL。~~ **已解决：采用 Stream 模式，钉钉 SDK 主动从内网发起长连接，不需要公网 IP / 域名 / 端口映射。**
说明	Stream 模式下，内网机器主动向钉钉云端发起 WebSocket 连接，消息通过该长连接推送，完全规避了公网可达性问题。~~方案 B（FRP/ngrok）~~ 不再需要。

维度	内容
状态	🟢 已解决

R-029 🟡 Agent 框架自研 — 缺少具体 Tool Calling 实现方案

维度	内容
涉及文档	04-系统概要设计 §5.4 (工具系统设计)、§7.7 (自研 vs LangChain 决策)
风险描述	04 §7.7 决定"Agent 框架自研"，不用 LangChain。但整个方案只给出了 Tool 的 Python 类定义和路由伪代码，未设计：(1) Tool Calling 的 Prompt 注入方式（如何把 Tool Schema 注入 System Prompt？用 JSON 还是 XML？）；(2) Agent 循环逻辑（LLM 返回 tool_call → 执行 → 注入结果 → 再推理，循环几次？max_iterations？）；(3) 并行 Tool Calling 支持（Supervisor 同时调多个工具）；(4) 错误恢复（Tool 执行失败后 Agent 如何决策？）。自研框架的实现复杂度被低估，可能占 S3-4 大量时间。
影响范围	Agent 核心能力实现时间、S3-4 进度
发生概率	高 — 自研 Agent 框架是全项目最复杂的编码任务
应对措施	1. **S2 末尾（Agent 详细设计文档中）必须明确** Tool Calling 的完整实现方案，包括 Prompt 模板、循环逻辑、max_iterations 2. 考虑用 vLLM 的原生 `tool_choice` 参数（Qwen3 系列支持 Function Calling），减少自研 Prompt 工程量 3. S3 Day 1 先做一个 **Minimal Agent Loop** 的 spike（技术验证），确认 Qwen3-30B-A3B 的 Tool Calling 能力是否足够
责任人	项目负责人
截止日期	S2 末尾（Agent 详设完成前）
状态	🟡 待处理

R-030 🟡 Qwen3-30B-A3B (MoE 3B 激活) 可能无法胜任多步 Tool Calling

维度	内容
涉及文档	04-系统概要设计 §4.2 (示例流程 7 步推理)、§5.1 (Supervisor 6 步决策)、§7.1 (模型选型)
风险描述	04 §4.2 的新建线索示例展示了 **7 步连续推理**（意图理解→实体提取→天眼查→评分→CRUD→钉钉推送→分配），§5.1 Supervisor 有 6 步决策流程。但实际执行模型 Qwen3-30B-A3B 只有 **3B 激活参数**，这意味着：(1) 多步 Tool Calling 中间可能"忘记"前面的工具输出；(2) 复杂意图理解（如"把这个月所有跟进不力的商机列出来并重新分配给张三"）可能无法正确拆解；(3) JSON 格式的 Function Call 输出可能格式不标准，导致工具调用失败。
影响范围	核心 Agent 功能可用性
发生概率	高 — 3B 激活参数的 Function Calling 能力有限
应对措施	1. **S3 Day 1-2 做 spike 测试**：用 10 个典型 CRM 场景测试 Qwen3-30B-A3B 的 Tool Calling 成功率 2. 如果成功率 < 80%，立即切换 vLLM A 到 **Qwen3.5-35B-A3B**（已有模型文件） 3. 降级方案：复杂多步任务走云端 API（脱敏后调用 Claude/Qwen-Max），简单单步走本地 vLLM 4. 在 Agent Loop 中增加 JSON 格式校验 + 自动修复层
责任人	项目负责人
截止日期	S3 Day 2 前
状态	🟡 待处理

R-031 🟡 PostgreSQL 主库在 macOS 上 — Docker 化困难

维度	内容
涉及文档	03-硬件清单 §2 (mac_min_8T macOS 15.1)、04-§6.1 (部署拓扑)、R-012 (mac_min_8T 角色过载)

维度	内容
风险描述	PostgreSQL 主库运行在 macOS 上（brew install），S7-8 要 Docker 化部署。但 macOS Docker Desktop：(1) 不支持 GPU 透传（如果后续 Embedding 放本机）；(2) Docker for Mac 的磁盘 I/O 性能比 Linux 原生差 30%-50%（使用 VirtioFS 后改善但仍不如原生）；(3) macOS 的 Docker 运行在 HyperVisor 虚拟化层，内存开销更大。生产数据库跑在 macOS Docker 上是非常规做法。
影响范围	S7-8 Docker 化迁移难度、生产数据库性能
发生概率	中 — S7-8 才暴露
应对措施	1. 一期生产 PG 继续用 macOS 原生 brew 安装，不入 Docker 2. S7-8 Docker compose 只容器化 CRM 后端和 Redis，PG 保持裸机 3. 长期方案：将 PG 主库迁移到 data_center (Ubuntu)，mac_min_8T 仅跑前端和应用层
责任人	项目负责人
截止日期	S6 末尾前确认方案
状态	🟡 待处理

R-032 🟡 Celery Worker 位置分散 — 跨网络消费 Redis Broker

维度	内容
涉及文档	04-系统概要设计 §6.1 (data_center 跑 Celery Worker)、§6.2 (Worker 连接 mac_min_8T Redis :6379)、07 S1-2 Day 8
风险描述	04 §6.2 清晰标注 Celery Worker 在 data_center (.14.90) 上，而 Redis Broker 在 mac_min_8T (.12.50) 上。这意味着每个任务的 fetch + ack 都要跨内网传输。如果内网交换机出现抖动或 Redis 在 macOS 上出现高延迟，Worker 会反复超时重连。同时 04 §7.3 提到 Redis 还承担 Stream 事件总线、对话上下文缓存等多重角色，已有单点压力。
影响范围	异步任务可靠性（报告生成、批量评分、数据同步）
发生概率	中 — 内网延迟通常低，但 macOS Redis 稳定性是变量
应对措施	1. 配置 Celery `broker_transport_options` 的 `socket_timeout` 和 `socket_connect_timeout` 2. 设置 `task_acks_late=True` 防止任务丢失 3. 考虑在 orchestrator 上部署一个 Celery Worker 副本作为冗余
责任人	项目负责人
截止日期	S3 中
状态	🟡 待处理

R-033 🟡 缺少错误处理和用户体验降级方案

维度	内容
涉及文档	04-系统概要设计 §2.3 (原则: "可降级")、06-§3.2 (降级策略矩阵)
风险描述	04 §2.3 提出"AI 不可用时，基础 CRUD 仍通过 Web 看板可操作"。但 06 §3.2 的降级策略只覆盖了 vLLM/云端/后端/数据库四层，未覆盖：(1) Agent 理解意图错误时的纠错交互（"你是不是想说…？"）；(2) 钉钉消息超时未回复时的用户体验（用户不知道 Agent 是在思考还是挂了）；(3) 创建操作失败后的重试/回滚策略（Agent 调了天眼查→评分→写DB，写 DB 失败，前面的操作如何回滚？）；(4) Web 看板的"降级入口"何时显示、如何切换。
影响范围	用户体验、操作可靠性
发生概率	必然 — LLM 的非确定性特征决定了错误不可避免
应对措施	1. Agent 回复中增加"thinking"状态推送（钉钉卡片显示"正在处理…"） 2. 设置 Agent 超时告警（>15s 未返回则推送"请稍候"）

维度	内容
	3. Tool 调用链增加事务补偿机制（或在 Supervisor 层做失败回滚） 4. 在 Agent 详细设计文档中明确错误交互场景
责任人	项目负责人
截止日期	S3 前（Agent 详设中定义）
状态	🟡 待处理

R-034 🟡 02 智能扩展需求与一期功能范围不对齐

维度	内容
涉及文档	02-智能扩展需求 (13 个大模块×50+ 小功能)、04-系统概要设计 §5.2 (14 Agent 规划)、06-总体开发计划 §2.1 (一期 6 Agent)
风险描述	02 文档列出了极其庞大的智能化需求（线索自动收集、竞品监测、拜访路线编排、周报自动生成、目标拆解、漏斗预警…），总计 50+ 项智能功能。04 规划了 14 个 Agent 来承接这些需求。但 06 一期只做 6 个 Agent。问题是：**02 文档中哪些功能属于一期？哪些属于二期？完全没有标注**。Orchestrator 的 DocParser 读取 02 文档时，可能将所有需求解析为一期任务，导致任务膨胀。
影响范围	Orchestrator 任务生成准确性、客户期望管理
发生概率	高 — DocParser 按文档生成任务，02 未标注分期
应对措施	1. **在 02 文档中为每项功能增加"分期"列**：一期 / 二期 / 三期 / 远期 2. 与客户确认一期必须交付的智能化功能清单 3. DocParser 配置中增加"ignore_phase > 1"过滤规则
责任人	项目负责人
截止日期	S3 前
状态	🟡 待处理

R-035 🔵 缺少 Agent 可观测性设计（Tracing / 推理链路可视化）

维度	内容
涉及文档	04-系统概要设计 §8.3 (AgentLog 表)、05 §4.5 (评估维度)、08 §6 (Dashboard)
风险描述	04 §8.3 设计了 `AgentLog` 表记录推理链路，但没有可视化和查询方案。当 Agent 行为异常时（如评分偏差、错误分配），需要能快速回放推理链路，定位是哪个 Tool 返回了错误数据、还是 LLM 推理出了错误判断。当前缺少：(1) 结构化 Trace ID 在各组件间的传播设计；(2) AgentLog 的查询/回放 UI；(3) 慢查询告警（Agent 推理 >10s 的 trace 自动标记）。
影响范围	Prompt 调优效率、生产问题排查速度
发生概率	必然 — Agent 系统必须有可观测性
应对措施	1. S3-4 实现 Agent 时同步实现 Trace ID 注入（贯穿 Supervisor → Tool → LLM 调用） 2. S7-8 Vue 看板中增加 AgentLog 回放页面 3. 利用现有 Prometheus + Grafana 追踪 Agent 调用成功率和延迟分布
责任人	项目负责人
截止日期	S3 中（与 Agent 实现同步）
状态	🔵 排入 Backlog

R-036 🔵 Git 裸仓库位置与文档不一致

维度	内容
涉及文档	07-Sprint任务卡 S1-2 Day 1 W4 ("在 mac_min_8T 上初始化 Git 裸仓库")、08-§1.3 ("Git + post-receive hook → mac_min_8T")、R-016 ("Git Bare Repo 位于 gateway 单机上")
风险描述	R-016 写的是"Git Bare Repo 位于 gateway 单机上"，但 07 S1-2 Day 1 和 08 §1.3-1.4 明确写的是 Git 裸仓库在 **mac_min_8T** 上。R-016 中的"gateway"可能是笔误，这会误导 Git 备份方案的实施。
影响范围	Git 备份策略正确性
发生概率	中 — 如果按 R-016 描述去 gateway 备份，会备份到错误位置
应对措施	修正 R-016 中"gateway"为"mac_min_8T"，统一描述
责任人	项目负责人
截止日期	立即
状态	🔵 待修正

6. 跨文档一致性问题

以下问题不影响系统运行，但会导致 Orchestrator 的 DocParser **解析出矛盾指令**，应在 S1 前统一修正。

C-001 测试执行位置描述不一致

维度	内容
涉及文档	05-测试方案 §2 vs §5.1-§7.2、08 §5.1、09 §2.4
问题	05 §2 已声明迁移到 orchestrator，但 §5-§7 多处仍写"L20"。08 §5.1 和 09 §2.4 已更新为 orchestrator。
修正建议	批量替换 05 §5-§7 中的 "L20" → "orchestrator 本地"。~~保留 aliyun-agent 作为 Staging 的引用~~ — aliyun-agent 已移除，Staging 迁至 orchestrator

C-002 vLLM 实例编号约定不一致

维度	内容
涉及文档	04 §7.1 vs 08 §2.5
问题	04 §7.1 原定义 3 个推理实例（A/B/C），现仅保留 vLLM A (4090 生产)。~~B=orchestrator 审查+测试~~ 已移除（AutoReview 迁云端 API），~~C=aliyun-agent 预发布~~ 已移除（aliyun-agent 整机移除）。
修正建议	已解决 — 04/08 已同步更新，仅保留 vLLM A 编号

C-003 post-receive hook 逻辑版本不一致

维度	内容
涉及文档	05-测试方案 §6.1 vs 09-测试机搭建计划 §2.2
问题	05 §6.1 的 hook 逻辑： `git push → SSH 到"测试机(L20)" → pytest` 。09 §2.2 的 hook 逻辑： `git push → webhook → orchestrator 调度` 。两个流程完全不同（直接 SSH 执行 vs webhook + orchestrator 调度）。
修正建议	以 09 的 webhook 方案为准（更符合 Orchestrator 架构），更新 05 §6.1

C-004 08 §2.5 YAML 缩进错误

维度	内容
涉及文档	08-自动化开发流水线.md §2.5
问题	§2.5 的 `aider` 配置 YAML 中，某些字段缩进不一致（混用 tab 和空格），可能导致 YAML 解析失败。
修正建议	统一为 2 空格缩进，逐行检查 §2.5 的 YAML 代码块

C-005 04 §6.1 缺少 orchestrator 角色

维度	内容
涉及文档	04-系统概要设计.md §6.1 部署拓扑
问题	04 §6.1 列出的部署拓扑中包含 4090、mac_min_8T、gateway 等机器角色，但未明确列出 orchestrator 的角色。orchestrator 是 08 文档引入的核心调度节点。
修正建议	在 04 §6.1 的部署表中补充 orchestrator 行：`orchestrator (.14.201) — Xeon 64C+128G — 薄调度层（任务编排）、Staging 预发布验收、自动测试执行`

C-006 Judge 评分维度不一致

维度	内容
涉及文档	05-测试方案 §4.5
问题	05 §4.5 定义了 5 个评分维度（准确性、完整性、一致性、响应时间、用户体验），而实际 judge.py 实现使用 4 个维度（准确性、相关性、完整性、专业性）。维度名称和数量不匹配。~~（原引用 09 §4 judge.py，09 已废弃。）~~
修正建议	统一为 judge.py 实现的 4 维度（代码为准），更新 05 §4.5 的文档描述

C-007 R-016 中 Git 裸仓库位置描述错误

维度	内容
涉及文档	本文档 R-016 vs 07-Sprint任务卡 S1-2 Day 1 W4、08 §1.3
问题	R-016 描述 Git Bare Repo 位于 **gateway** 上，但 07 S1-2 Day 1 W4 和 08 §1.3-1.4 明确写在 **mac_min_8T** 上。R-016 的"gateway"为笔误。
修正建议	R-016 中"gateway"修正为"mac_min_8T"（另见 R-036）

C-008 02 智能扩展需求缺少一期/二期分期标注

维度	内容
涉及文档	02-智能扩展需求.md 全文 vs 04-§5.2 (Agent 分期)
问题	02 文档列出 13 个大模块、50+ 项智能功能，但**无一标注属于哪个交付阶段**。04 §5.2 将 14 Agent 按 Phase 1/2/3 规划，但 02 的需求与 04 的 Phase 对应关系不明确。Orchestrator DocParser 解析 02 时会将所有需求视为同等优先级。
修正建议	在 02 文档每节增加「分期：一期 / 二期 / 远期」标签，与 04 §5.2 Agent Phase 对齐

7. 优先行动计划与跟踪表

7.1 S1 前（即刻行动）

序号	行动项	关联风险	预计耗时	状态
1	✅ 将所有 API Key / 密码迁移到 `.env` 文件，文档中替换为占位符	R-001	2h	✅
2	✅ 修复 Qwen3-30B-A3B 模型缺失 shard 11/16（4090 已运行）	R-002	1h (下载)	✅
3	✅ Orchestrator 重新定位为薄调度层（aider 已覆盖核心编码循环，自举悖论消解）	R-003	—	✅
4	✅ 批量修正 05 文档中 "L20" 引用 → "orchestrator"（30+ 处修正，05 版本升至 v0.5）	R-004, C-001	1h	✅
5	确认功能砍减方案（核心 3 Agent vs 6 Agent）	R-005, R-022	1h	⬜
6	准备 yibuapi.com fallback 方案 + 配置备用编码模型	R-006	1h	⬜
7	✅ ~~确认 aliyun-agent 到内网的网络方案~~ — 已解决（aliyun-agent 已移除，全部业务内网运行）	R-007	—	✅
8	✅ ~~发起钉钉开发者应用注册~~ — 已完成（Stream 模式机器人已创建，内网已打通）	R-010	—	✅
9	✅ 确认 AutoReview 模型策略 → 云端 API（已决策，同步移除 vLLM B）	R-009, R-018	0.5h	✅
10	修复跨文档一致性问题 C-001 ~ C-008	C-001~C-008	3h	⬜
25	**与客户确认一期 ERP(U9) 对接范围**	R-024	2h	⬜
26	**与客户确认多语言/国际地图是否一期**	R-025	1h	⬜
27	✅ ~~确认 gateway 公网可达性 + SSL 方案~~ — 已解决（钉钉采用 Stream 模式，无需公网/SSL）	R-027, R-028	—	✅
28	**在 02 文档中标注每项功能的交付分期**	R-034, C-008	3h	⬜
29	**制定数据安全合规方案（PIPL/数据分类）**	R-023	4h	⬜

7.2 S1-S2 期间

序号	行动项	关联风险	预计耗时	状态
11	编写数据库设计文档	R-011	4h	⬜
12	配置 Redis AOF 持久化	R-015	0.5h	⬜
13	设置 Git Bare Repo 每日备份	R-016	1h	⬜
14	基础 docker-compose.yml（PostgreSQL + Redis）	R-020	2h	⬜
15	~~实现最简 Orchestrator Dashboard~~ → 钉钉交互查询(~2h) + 终端看板(~1h)；Web Dashboard 列为可选待办	R-019	3h	⬜
16	磁盘空间监控 cron	R-021	0.5h	⬜
17	✅ ~~共享文件分布式锁方案~~ — 不需要：采用串行提交 + 共享文件单点管理	R-013	—	✅
30	**编写 Tool Calling 完整实现方案**	R-029	4h	⬜

7.3 S2 末尾

序号	行动项	关联风险	预计耗时	状态
18	编写 Agent 详细设计文档（含 Tool Calling 循环、错误恢复）	R-011, R-029, R-033	8h	⬜
19	编写钉钉集成方案文档（含 SSL/公网方案）	R-011, R-027, R-028	4h	⬜

序号	行动项	关联风险	预计耗时	状态
20	~~升级 vLLM B~~ 已废弃（vLLM B 已移除）	R-018	—	~~—~~

7.4 S3 前

序号	行动项	关联风险	预计耗时	状态
21	设置 Cloud Judge API 频率上限 + 成本监控	R-017	1h	⬜
22	确认钉钉 AppKey/AppSecret 到位	R-010	(依赖审批)	⬜
31	**Spike 测试 Qwen3-30B-A3B Tool Calling 能力（10 场景）**	R-030	4h	⬜
32	**设计 Redis Stream Consumer Group + MAXLEN + Dead Letter**	R-026	2h	⬜
33	**配置 Celery broker_transport_options 超时 + ack_late**	R-032	1h	⬜

7.5 S5 前

序号	行动项	关联风险	预计耗时	状态
23	✅ ~~确认 Embedding 模型部署方案~~ — 已决策：一期阿里云百炼 Embedding API，二期视量迁本地 BGE-M3	R-008	—	✅

7.6 S6 末尾

序号	行动项	关联风险	预计耗时	状态
24	编写部署方案文档（含 macOS PG 不入 Docker 决策）	R-011, R-031	4h	⬜

7.7 S7-S8 期间（v1.1 新增）

序号	行动项	关联风险	预计耗时	状态
34	**Vue 看板增加 AgentLog 推理链路回放页面**	R-035	4h	⬜
35	**Grafana 面板增加 Agent 调用成功率/延迟分布**	R-035	2h	⬜

8. 风险全景统计

等级	数量	原有	v1.1 新增	状态
🔴 P0 阻塞	3	3	—	3 已解决 / 0 待处理
🟠 P1 高风险	11	8	+3 (R-023, R-024, R-025)	5 已解决 (R-004, R-007, R-008, R-009, R-010) / 6 待处理
🟡 P2 中风险	15	6	+9 (R-026 ~ R-034)	3 已解决 (R-013, R-027, R-028) / 12 待处理
🔵 P3 建议	7	5	+2 (R-035, R-036)	1 已废弃 (R-018) / 2 已缓解 (R-019, R-020) / 4 待处理
⚪ 一致性	8	6	+2 (C-007, C-008)	0 已修正 / 8 待修正
合计	**44**	**28**	**+16**	**14 已解决 / 1 已废弃 / 29 待处理**

> **v1.1 Review 变更摘要**：

- 新增 14 项风险（R-023 ~ R-036）：覆盖数据合规(PIPL)、ERP分期确认、多语言/国际化、事件总线设计、SSL/HTTPS、公网可达性、Agent框架自研、模型 Tool Calling 能力、macOS PG Docker化、Celery跨网络、错误处理降级、需求分期对齐、Agent可观测性、Git位置修正
- 新增 2 项一致性问题（C-007, C-008）
- 行动计划新增 11 项（#25 ~ #35）
- 风险总数从 28 → 44

v1.2 架构简化变更：

- 移除 vLLM B (orchestrator)：AutoReview/DocParser 改用云端 API，Agent 集成测试推理统一调用 4090 vLLM A
- R-003 状态 → 🟢 已解决：aider 已覆盖核心编码循环（上下文读取→代码生成→写入→commit→测试→自动修复→重试），Orchestrator 从"重型定制系统"重新定位为"薄调度层"（~200 行脚本），自举悖论消解
- R-009 状态 → 🟢 已解决（迁移云端 API）
- R-018 状态 → 已废弃（vLLM B 不再存在）
- R-002 影响范围缩小为 vLLM A + vLLM C（不再包含 vLLM B）
- 04/06/08 文档同步更新：vLLM B 引用移除 + Orchestrator 描述简化为薄调度层

v1.3 aliyun-agent 移除变更：

- R-007 状态 → 🟢 已解决：不再打通云端↔内网网络，直接移除 aliyun-agent，全部业务在 5 台内网机器上运行
- Staging 预发布验收迁至 orchestrator（独立 DB + Redis DB 2 + 独立配置）
- vLLM C 移除，Agent 推理统一调用 4090 vLLM A
- 文档 03～09 已全部同步更新，09 整篇废弃
- R-001/R-002/R-005/R-019 中 "6 台" → "5 台"，vLLM 实例数同步缩减
- R-028 应对措施移除 aliyun-agent 方案，C-001/C-002 同步更新
- 行动计划 #7 标记 ✅ 完成，统计数据同步更新

v1.4 Embedding 方案决策：

- R-008 状态 → 🟢 已解决：一期使用阿里云百炼 Embedding API（text-embedding-v3），零部署零 GPU，成本极低
- 二期预留：知识量达万级时迁移到本地 BGE-M3（4090 与 vLLM A 并存）
- Knowledge 表 vector(1024) 设计不变
- 04 §7.1 Embedding 选型同步更新

v1.5 钉钉集成确认（Stream 模式）：

- R-010 状态 → 🟢 已解决：钉钉开发者应用已创建，采用 Stream 模式（长连接），内网已打通可获取数据
- R-027 状态 → 🟢 已解决：Stream 模式无需 HTTPS 回调 URL，SSL 不再是钉钉集成阻塞项
- R-028 状态 → 🟢 已解决：Stream 模式无需公网 IP/域名/端口映射，内网机器主动发起 WebSocket 连接
- 行动计划 #8、#27 标记 ✅

v1.6 Git 冲突风险重新评估：

- R-013 状态 → 🟢 已缓解：原方案（分布式锁 + 自动 merge）过度工程，1 人团队采用串行提交 + 人工确认 + 共享文件单点管理即可
- R-012 描述修正：移除过时的 "Ollama Embedding 推理" 职责（R-008 已决议一期走云端 API，二期部署在 4090）
- 08 §5.2 push 流程更新：移除自动 rebase/GitConflictError，改为通知人工确认
- 08 §5.4 分支策略说明更新：明确串行提交 + 共享文件单点管理策略
- 行动计划 #17 标记 ✅ （分布式锁不再需要）

v1.6.3 R-020 Docker Compose 已确认：

- R-020 状态 → 🟢 已采纳：S1 编写基础 docker-compose.yml（PostgreSQL 16 + pgvector + Redis 7），开发期间 Docker 跑依赖服务，应用裸机开发
- 行动计划 #14 不变（已有 2h 预估），docker-compose.yml 纳入 Git 仓库

v1.6.2 R-019 可观测方案决策：

- R-019 状态 → 🟢 已缓解：原方案（FastAPI Web Dashboard 4h）替换为三层渐进式可观测
- 层级 1：钉钉交互式查询（`状态` / `机器` / `日志` / `卡住` 指令），复用 Stream 机器人，~2h

> - 层级 2：终端实时看板（live_status.py + watch），~1h
> - 层级 3：Web Dashboard 降级为可选待办（S7-S8，AI 辅助 ~30min）
> - 行动计划 #15 更新：4h → 3h，内容调整为层级 1+2
>
> **v1.6.1 过时引用全量清理**：
>
> - R-014：移除"09 §2.5 vLLM 实例 B (orchestrator)"引用（09 已废弃、vLLM B 已移除），Orchestrator 描述更新为"薄调度层"，检查清单更新为 vLLM A + 钉钉 Stream
> - R-015：涉及文档移除"09-测试机搭建计划.md"引用，Redis 职责补充 Stream 事件总线
> - R-016：Git Bare Repo 位置从"gateway（09 §2.2）"修正为"mac_min_8T（08 §5.3）"
> - R-017：涉及文档移除"09 §4 judge.py"引用
> - R-021：磁盘描述移除"3.5-35B 约 28GB"（vLLM B 模型已不存在），聚焦 4090 + mac_min_8T
> - C-005：orchestrator 角色从"调度中心 + vLLM B + 测试执行"修正为"薄调度层 + Staging + 自动测试"
> - C-006：移除"09-测试机搭建计划 §4 judge.py"引用，标注 09 已废弃

> **文档维护说明**：每解决一项风险，请更新对应的"状态"字段（⬜→✅ 或 🔴→🟢），并在行动计划表中标记完成。建议每个 Sprint 结束时对本文档做一次全面 Review。